

# Project Report

## Geometry Dash

ENSC 452 - Advanced Digital Systems Design

Group 5

Artin Jafari Rad - 301555369

Ali-Aryo Otufat-Shamsi (Ali) - 301553709

|                                                                          |           |
|--------------------------------------------------------------------------|-----------|
| <b>1.0 Introduction.....</b>                                             | <b>3</b>  |
| <b>2.0 Background.....</b>                                               | <b>4</b>  |
| <b>3.0 System Overview.....</b>                                          | <b>5</b>  |
| 3.1 Requirements.....                                                    | 5         |
| 3.2 Preferences.....                                                     | 7         |
| 3.3 Constraints.....                                                     | 7         |
| 3.4 System Block.....                                                    | 8         |
| 3.5 Use-Case Diagram.....                                                | 8         |
| 3.6 Algorithm Flowchart.....                                             | 9         |
| <b>4.0 Outcomes.....</b>                                                 | <b>11</b> |
| 4.1 Fulfillment of Initial Requirements.....                             | 11        |
| 4.2 Realization of Secondary and Polish Objectives.....                  | 11        |
| 4.3 Architectural Pivots and Design Evolution.....                       | 14        |
| 4.4 Final System Synthesis.....                                          | 15        |
| <b>5.0 Description of Blocks(HW/SW).....</b>                             | <b>16</b> |
| <b>5.1 Hardware Blocks.....</b>                                          | <b>16</b> |
| 5.1.1 Custom VGA Controller Integration.....                             | 17        |
| 5.1.2 I2S Audio Codec Configuration.....                                 | 17        |
| 5.1.3 Linear Feedback Shift Register (LFSR) IP for RNG.....              | 18        |
| 5.1.4 Audio Threshold Detector (ATD) IP.....                             | 19        |
| 5.2 Software Subsystems.....                                             | 20        |
| 5.2.1 Dual-Core Orchestration & Mailbox Synchronization.....             | 20        |
| 5.2.2 Core 0: Game Engine, Physics, and Collision Detection.....         | 21        |
| 5.2.3 Core 0: Math-Based Dynamic Video Driver (Replacing SD Images)..... | 22        |
| 5.2.4 Core 1: SD Card Audio Streaming & Cache Management.....            | 23        |
| <b>6.0 Implementation.....</b>                                           | <b>24</b> |
| 6.1 Design Methodology.....                                              | 24        |
| 6.2 Milestones and Timeline.....                                         | 25        |
| 6.3 Design Changes.....                                                  | 31        |

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>7.0 Difficulty &amp; Project Complexity.....</b> | <b>32</b> |
| <b>8.0 References.....</b>                          | <b>33</b> |

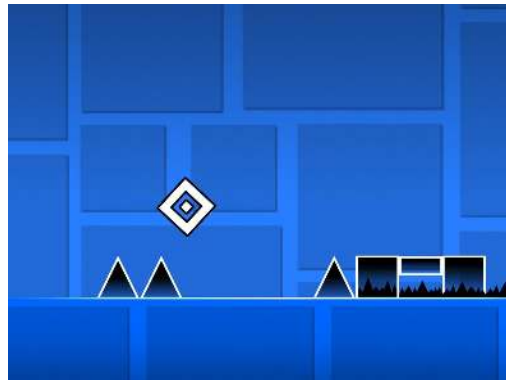
# 1.0 Introduction

Our goal for ENSC 452 is to implement a project on the Xilinx Zynq-7020 Development Board. By leveraging the unique architecture of the Zynq System-on-Chip (SoC), we aim to seamlessly integrate high-performance ARM Cortex-A9 software processing with custom Programmable Logic (PL) hardware. Our group has chosen to create Geometry Dash, a rhythm-based action platformer where players navigate music-synced levels filled with obstacles like spikes by tapping to jump, to reach the end of each level while mastering timing and repetition to survive. Replicating this fast-paced arcade experience on a bare-metal architecture requires strict real-time performance, specifically maintaining a flawless 60 frames-per-second (FPS) video output and uninterrupted I2S audio streaming to guarantee perfect rhythm synchronization. This game will allow us to utilize different hardware components, both custom and IP cores, and dive deep into software to create the high-level design of our game. Specifically, the system utilizes a custom VGA video controller, a hardware Linear Feedback Shift Register (LFSR) for pseudo-random obstacle generation, and a real-time Audio Threshold Detector IP. To orchestrate these hardware blocks without AXI bus bottlenecks, we developed a dual-core software architecture linked by a shared-memory mailbox. The design responds to user input through a button on the FPGA, which triggers jump movement. Core 0 acts as the master processor, handling these hardware interrupts alongside dynamic, math-based C++ graphics rendering, while Core 1 acts as an isolated multimedia engine streaming massive WAV files directly from an integrated SD card. The game logic detects Game Over conditions, such as obstacle collisions, and manages the physics of the game. Ultimately, by bridging these low-level PL modules with robust software state machines, the final product delivers a complete, multi-mode gaming experience featuring standard, endless, and audio-reactive levels that fully demonstrate the capabilities of the Zynq hardware-software development stack.

## 2.0 Background

Geometry Dash is a classic mobile game released in 2013. Developed by RobTop Games, it is characterized by its simplistic one-touch controls and notoriously steep difficulty curve. The game has since become one of the most popular mobile games ever, and soon after got a PC port. At its core, the gameplay loop strips away complex mechanics in favour of pure reaction time and spatial awareness. The game is an auto side-scroller platforming game, which has the user jump over obstacles as they move towards the player's 'icon' (character). Because the scrolling speed is locked and cannot be slowed down by the user, the environment continuously forces rapid decision-making. These obstacles primarily consist of sharp triangles acting as spikes, solid walls, and bottomless pits that must be cleared with perfect vertical spacing. The game's levels are often synced with music, and taps may be lined up with certain beats. This rhythmic integration transforms the experience from a standard platformer into a highly engaging audio-visual challenge, where the electronic soundtrack serves as an auditory guide for upcoming hazards. The player has a selection of pre-made levels with various difficulties, and must practice these levels to be able to complete them in 'one attempt' (start to end without hitting Game Over conditions). There are absolutely no checkpoints within a standard level, creating a high-stakes environment where consistency is just as important as reflexes. If the hitbox of the player's icon collides with an illegal obstacle, the Game Over logic will trigger, and the level will restart. This immediate, punishing reset loop forces players to rely entirely on muscle memory and strict pattern recognition to progress. The goal is to make it to the end of the level without colliding with any illegal obstacles. Successfully reaching this finish line requires a perfect sequence of inputs, which is exactly the rigid, timing-critical environment we aim to replicate in hardware. Our game will utilize an FPGA button to allow the player's icon to jump. By relying on a bare-metal architecture, we can ensure that this input is processed with near-zero latency, avoiding the software overhead found in traditional operating systems. We will use the VGA port and protocol to display our game on a monitor. To match the fluid motion of the original game, our hardware must consistently drive the VGA output at a strict 60 frames per second without stuttering or screen tearing. Future improvements should include a pause menu to trigger when a specific FPGA button is pressed. Ultimately, understanding this foundational loop

of auto-scrolling, instantaneous death, and audio synchronization provides the necessary context for the hardware constraints and custom IP design choices discussed in the following sections.



## 3.0 System Overview

This section will go over a prioritized list of requirements, preferences, and constraints. Furthermore, how the user interacts with the game, the control flow, and the hardware that needs to be designed and implemented will be covered in this section.

### 3.1 Requirements

| Priority | Requirement                                                                                                                |
|----------|----------------------------------------------------------------------------------------------------------------------------|
| High     | A stable VGA connection and signal capable of maintaining a continuous 60 frames-per-second output without screen tearing. |
| High     | Implement basic gravity and jump physics that correctly calculate vertical velocity and collision hitboxes in real-time.   |
| High     | Use pushbutton interrupts to have the icon jump.                                                                           |
| High     | Render the play icon and obstacle spikes, and use mathematical pixel drawing to update pixels dynamically.                 |

| Priority | Requirement                                                                                                     |
|----------|-----------------------------------------------------------------------------------------------------------------|
| High     | Have a complete level with a hardcoded set of obstacles in an array to be as the map for the gameplay loop.     |
| High     | Play the correct audio corresponding to the level selected. Use I2S communication with the custom audio codec.  |
| High     | Accurate collision detection (Game Over detection) that stops and resets gameplay.                              |
| High     | A continuous, auto-scrolling background that smoothly updates memory pointers to simulate forward movement.     |
| High     | Implement a dual-core architecture to separate audio streaming from everything video and game logic related.    |
| Medium   | Audio synchronization with gameplay ensures that lines up obstacles spawn accurately on the beats of the music. |
| Medium   | Multiple selectable levels, including an Endless Mode, Audio mode, and standard mode.                           |
| Medium   | Customizable player sprites that can be selected from an interactive custom menu.                               |
| Low      | Start menu/pause menu featuring switch-based interrupts. Add visual state indicators.                           |
| Low      | More obstacle types: ramp, orb, etc., to increase the complexity and variety of the platforming sequences.      |

***Table 1: Prioritized System Requirements***

## 3.2 Preferences

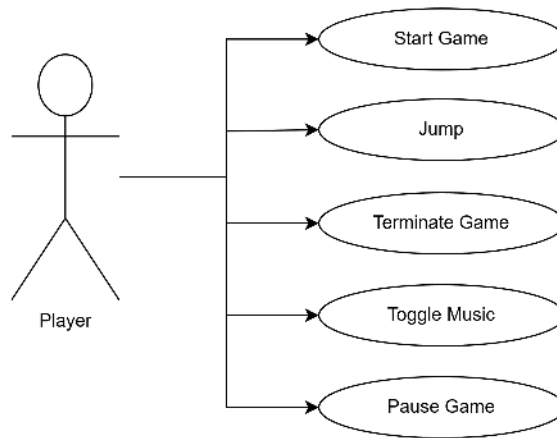
| Preference Description                                                                                                                                  |
|---------------------------------------------------------------------------------------------------------------------------------------------------------|
| Smooth and appealing gameplay featuring colour palettes and custom menu rendering instead of static memcopy images.                                     |
| Clear and non-fuzzy obstacles with working hitboxes to make gameplay fair and predictable.                                                              |
| Audio that is synchronized with the gameplay rhythm, using hardware threshold detection to read musical peaks and generate a level based on that music. |
| Modular design to make it easy to add new levels or new additions later.                                                                                |

*Table 2: System Preferences*

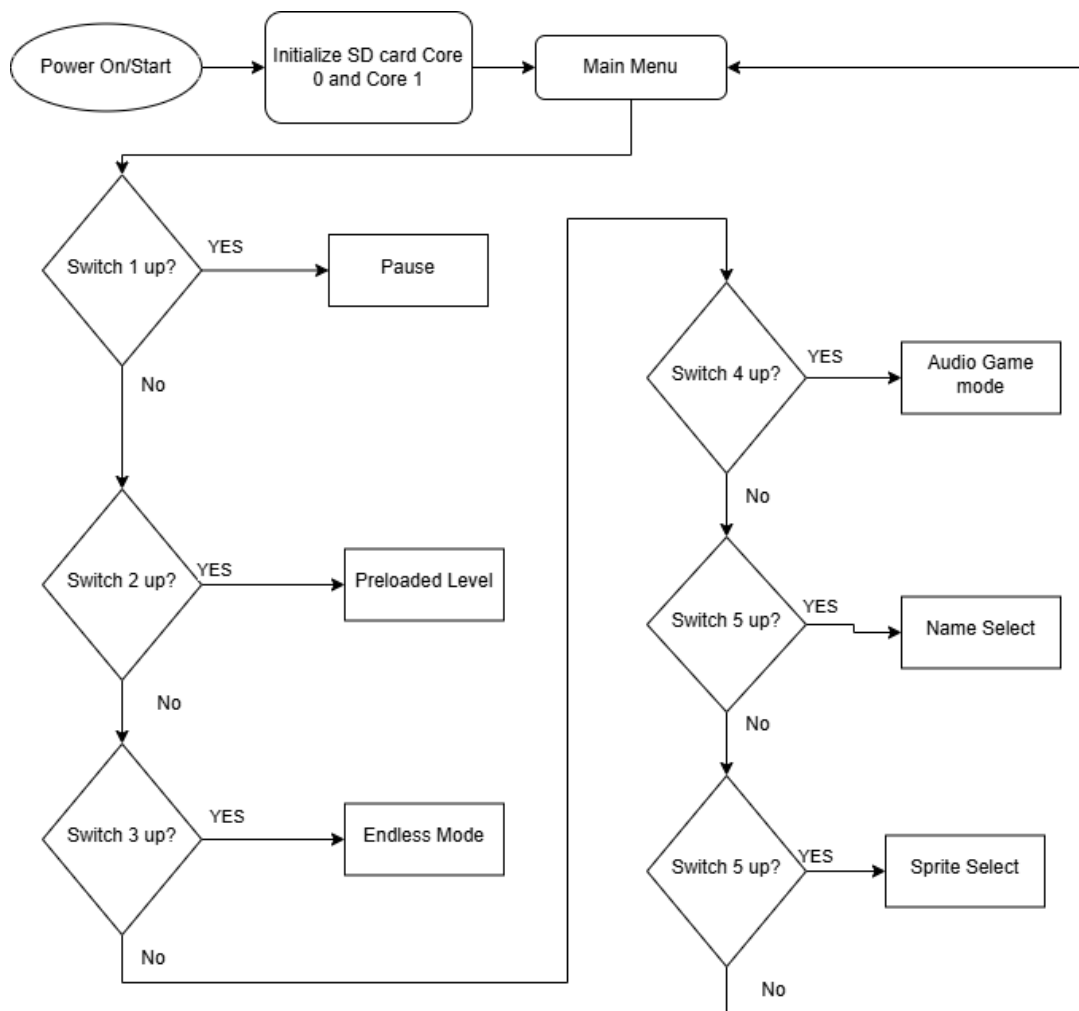
## 3.3 Constraints

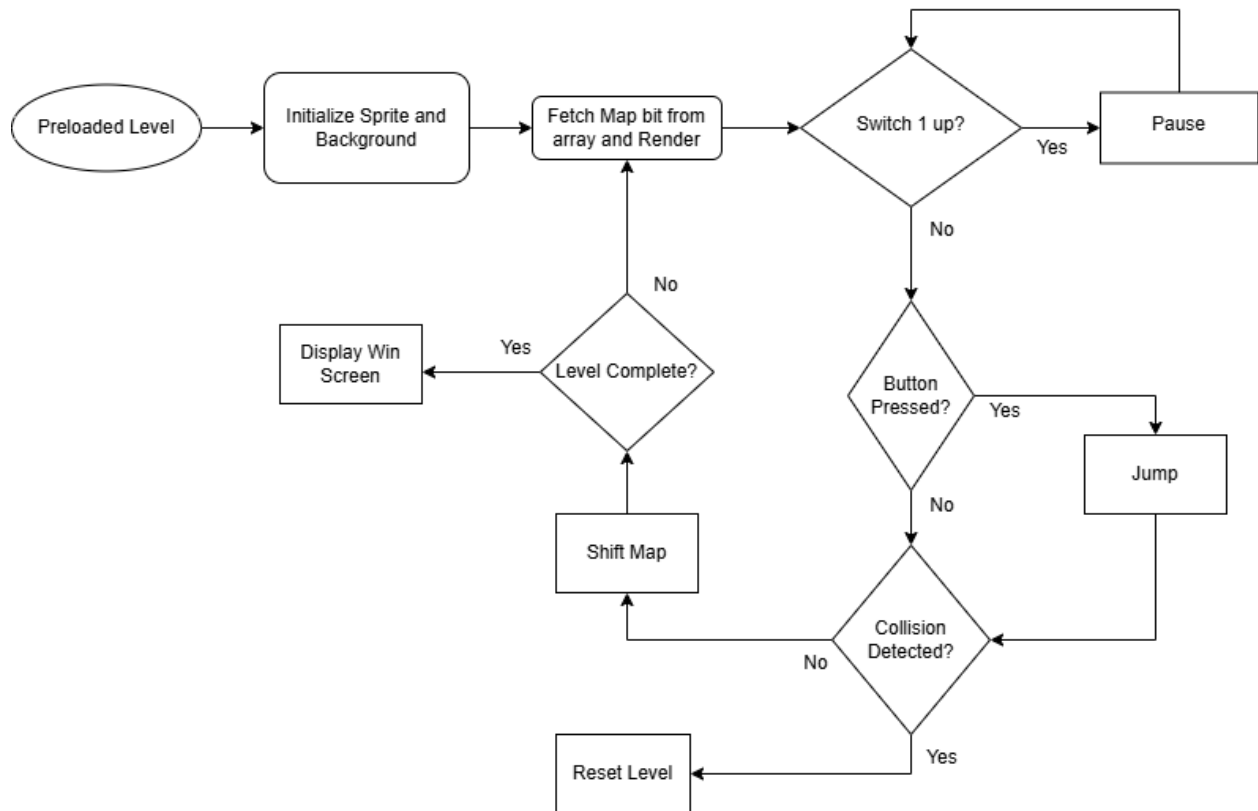
| Constraint Description                                                                                                                                    |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------|
| Push buttons may not be as responsive as we think, or may need robust software debouncing logic to prevent accidentally pressing too many times in a row. |
| Memory constraints regarding the size of the On-Chip Memory and DDR RAM when storing large WAV files for music.                                           |
| Cache size and management when loading the level. Making sure Core 1 reads the most accurate audio data.                                                  |
| VGA timing requirements that ensure the video driver calculates and updates the frame buffer well within the 16.6-millisecond window of a 60Hz display.   |
| Audio syncing with gameplay needs AXI bus management to prevent video rendering from being out of time with the audio codec of data.                      |





### 3.6 Algorithm Flowchart





## 4.0 Outcomes

We based our outcomes on a milestone system. We set dated milestones throughout the four-month development phase of this project.

### 4.1 Fulfillment of Initial Requirements

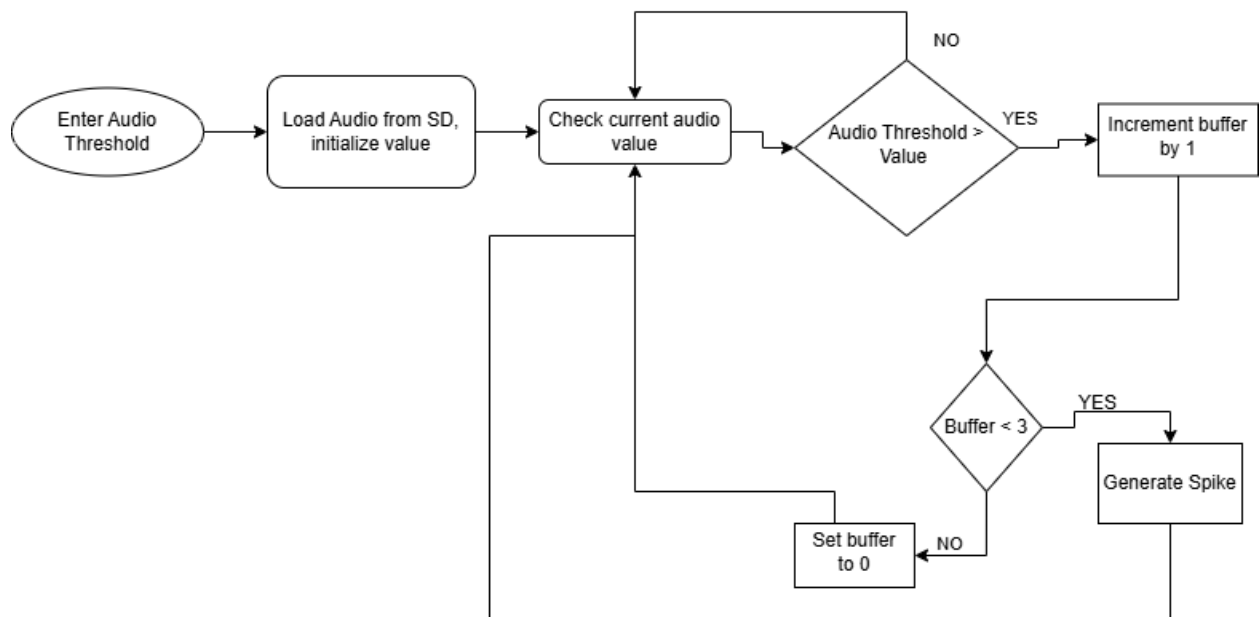
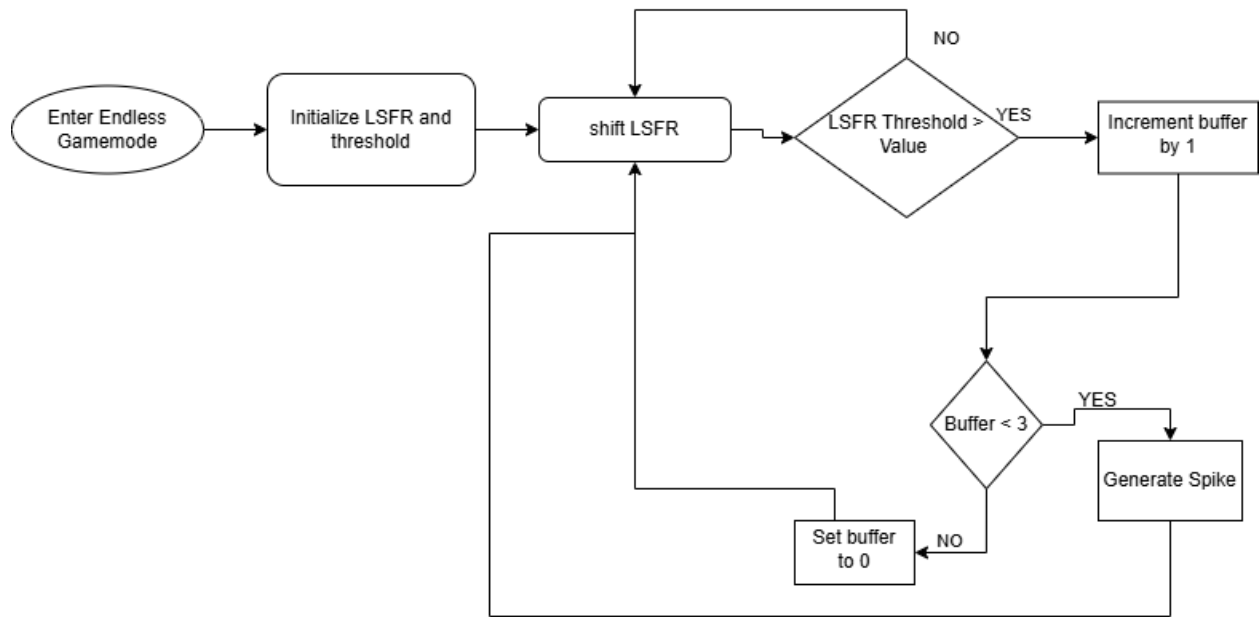
Looking back to our initial proposal against the final delivered game, we successfully met all of our high-priority requirements: we created a stable 60 FPS auto-scroller game with GIPO push-button interrupts for jump logic and pixel math for collision logic. The underlying physics engine accurately facilitates real-time calculations for gravity, vertical velocity, and instantaneous collision detection (triggering Game Over states). Player input is managed through hardware push-buttons that trigger a highly responsive jump mechanic by leveraging the low-latency characteristics of the bare metal environment. Finally, the audio playback subsystem remains precisely synchronized with the active gameplay state, fulfilling our core objective of merging high-performance video and audio components.

### 4.2 Realization of Secondary and Polish Objectives

There have been many extra features implemented in our project that elevate the final version's complexity. There are three different game modes to be played: a pre-set level (like the original mobile game), an endless mode that has the spikes being generated by the LFSR. The 8-bit LFSR generates a number from 1-255 (if a 0 is generated, the LFSR resets to generate a non-zero number). If the number generated is greater than a set threshold, a spike gets printed to the VGA. Multiple spikes can be generated back-to-back - up to a max of three spikes in a row, then a manual buffer will accrue before another spike can be generated. High scores are also tracked on local memory and displayed on the VGA: all-time high score (for that session), and last run's score.

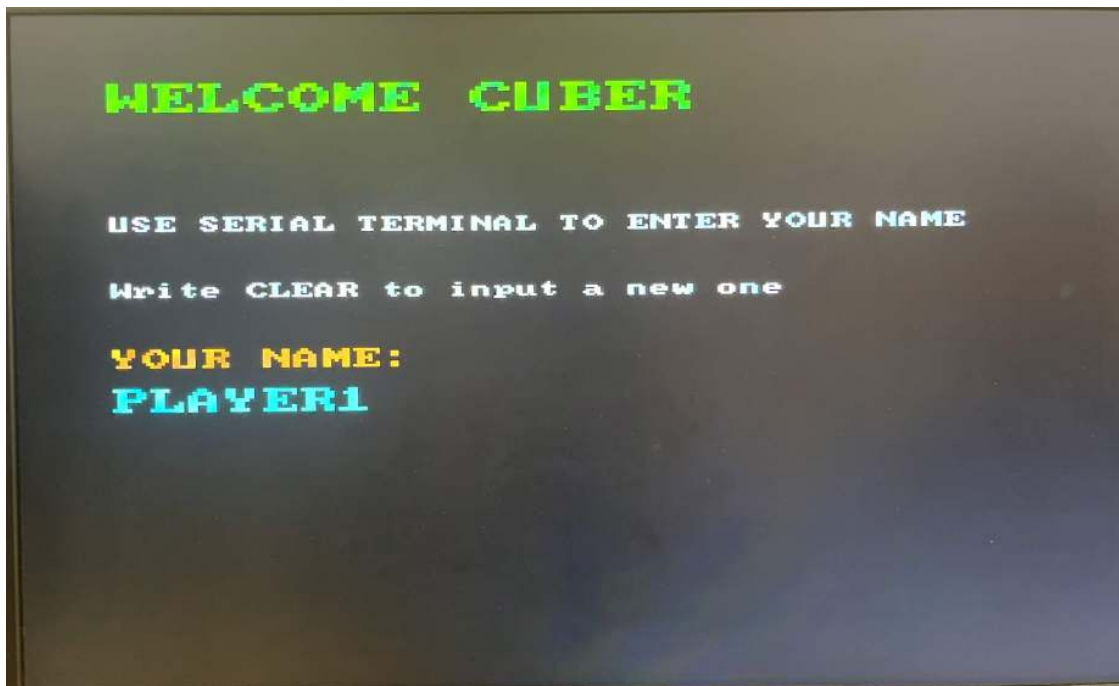


The last gamemode level is generated based on audio. Using the audio threshold detector custom hardware, it takes an input from the audio .WAV file from the SD card. If the peak of this audio file is greater than the preset threshold, a spike is generated. Again, a max of three spikes is allowed to be generated in a row to keep the game from becoming impossible. Because of the way this level is generated, the spikes are in sync with the music's beat hits. A custom menu state was created to allow the player to select a custom sprite for their playable icon. This menu was custom-made, designed, and pixel-generated. Using the same push button as the 'jump', the user can cycle through a pre-set amount of custom sprites to pick from.



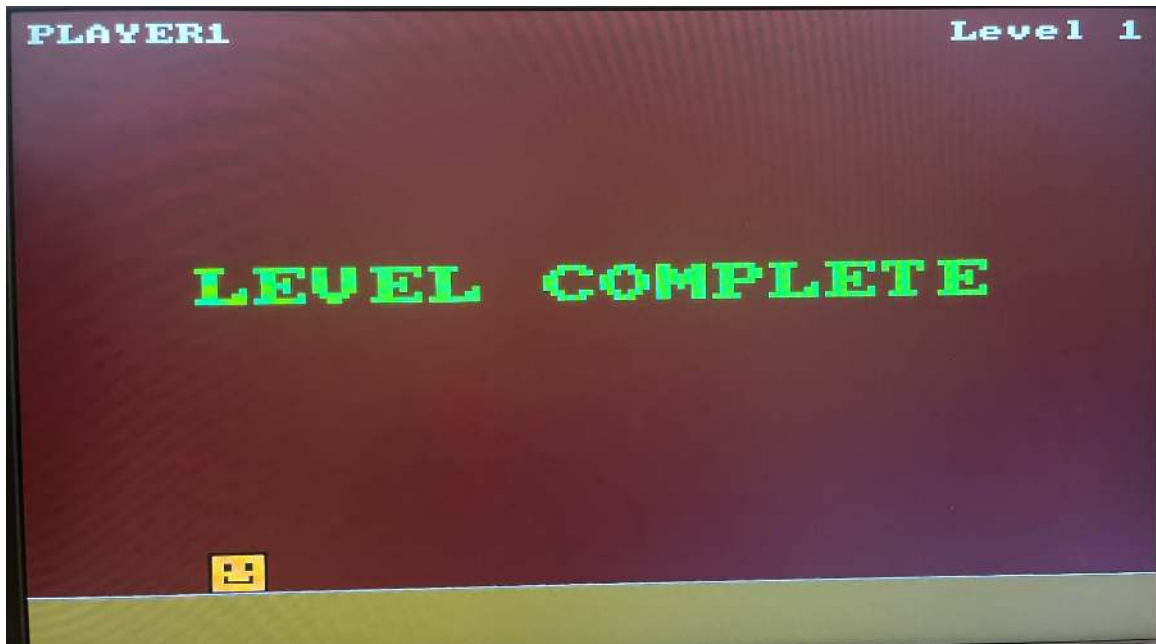
## 4.3 Architectural Pivots and Design Evolution

Initially, the plan was to store everything in the FPGA memory; however, the size of the WAV files forced a pivot to an SD card. Core 1 handles loading the WAV files from the SD card onto the FPGA memory. Another change was shifting away from using memcpy and .data files for our menus, to creating custom pixel generated menus for the main menu and pause screen. The main menu used some simple geometry to create a custom background, then called the draw sprite function to draw a sample of sprites to the main. Using an open source 8x8 ASCII library, paired with a custom write text function, the title FPA Geometry Dash is displayed on the main menu. The pause menu simply freezes the VGA, overlays a grey tint, then writes a simple pause icon in the center of the screen. These custom menus fit the theme of the game far better than a standard .data file off the internet would have.



## 4.4 Final System Synthesis

The final product is a highly responsive, dual-core, hardware-accelerated rhythm platformer that demonstrates the abilities of the Zynq-7020 SoC, as well as the hard work we have put into this project this semester. By offloading computationally heavy tasks, including PRNG and audio peak detection, to custom hardware IP blocks on the Programmable Logic, and orchestrating tasks across both ARM processors, the game achieves a flawless 60 FPS performance with uninterrupted multimedia synchronization.



## 5.0 Description of Blocks(HW/SW)

### 5.1 Hardware Blocks

For our version of Geometry Dash, we implemented two different pieces of custom hardware IP to handle high computational tasks. For our two custom pieces of hardware IP, we decided to create a Linear Feedback Shift Register (LFSR), as well as an Audio Threshold Detector. These blocks were integrated with the rest of our system using the AXI-4 Lite interface to connect to the Zynq chip. This AXI bus was the one recommended in class. These custom hardware blocks got combined with our origins A/V hardware that we put together in Vivado in the first few weeks of development.

Our baseline hardware was created by merging the two original Vivado tutorials provided by the professor. The VGA hardware and Audio codec were combined early in the development cycle to create a canvas for us to continue adding custom hardware.

The LFSR was used as a pseudo-random number generator. One of our requirements was to create an endless mode, where spikes would randomly and endlessly spawn in. This 8-bit LFSR would generate an output, which would be compared to a certain threshold in the software to determine if and how many spikes would be generated to the screen. Moving this logic to the programmable fabric made sure that the number generation would not interfere with anything else on the main loop, as doing too much polling on a single core can reduce the core's efficiency. This way, we have a constant and sustainable stream of numbers being generated to create our endless map.

The Audio Threshold Detector was the block needed to fulfill our requirement of having an 'audio-generated mode'. This means we wanted the spikes to be generated based on the input music. When the sound wave's amplitude spikes above a certain threshold (comparison happened on the hardware block here), a spike would be generated. This game mode allows us to create custom and unique maps based on the input song that is fed to this custom hardware block.

### 5.1.1 Custom VGA Controller Integration

The custom VGA controller was needed as video is a primary feature of Geometry Dash. The VGA controller was responsible for reading data from memory and writing that data to the monitor using the VGA protocol. The VGA controller hardware was created based on the VGA tutorial provided to the class.

This piece of hardware is connected to the Zynq using the AXI bus and interfaces with the Zynq's HP0 slave port. This makes the controller a master IP. This allows the controller to get image data from the DDR without causing traffic on the general-purpose AXI interconnect.

Our VGA system was configured for a 1280x1024 aspect ratio, running at a 60Hz refresh rate. This is a very reasonable standard for a 2D game like Geometry Dash. To configure these settings, a Clocking Wizard IP was used to provide a dedicated pixel clock that clocked at 108 MHz.

The VGA and ZedBoard use 12-bit RRGGBB (Red Green Blue) channels. These signals are located on the Horizontal Sync and Vertical Sync, and are mapped to the external pins by the boards consttriana file, to control the monitor interface. All of these configurations were listed on an internet source given to the class (check citations for details).

### 5.1.2 I2S Audio Codec Configuration

Arguably more important than video is audio for the game Geometry Dash, making this piece of hardware a must in terms of importance to integrate correctly and early on in the development cycle. We used and integrated the ADAU1761 audio codec on the ZedBoar; we did this by leveraging the tutorial provided to the class by the professor. This block allows streaming of audio to remain uninterrupted.

The audio codec is confused when using an I2C interface with the processing system. When we first boot the FPGA, the services of control registers write to initialize the codec. The audio codec sampling rate is set to 48 kHz.

The raw audio data was transmitted from the programmable logic to the audio coded using the I2S protocol. An I2S transmitter IP was used to convert parallel audio samples from the AXI bus into a usable bitstream, required by the audio codec. This setup ensures we have a low-latency path for the 24-bit audio samples to be processed by the processing system.

Another key requirement that we needed to lock down was the synchronization and clocking of the audio subsystem. This was done by giving a dedicated clock generation within the PL to the audio subsystem. This clock is in synch with the I2S master clock required by the codec. This prevents audio from distorting or losing quality during long play sessions.

For the real implementation into our full system, we focus exclusively on the audio path to maximize the bus sandwich for optimal streaming. We dedicated Core 1 to do nothing but poll the audio hardware to make sure our audio streaming was as close to perfect as possible.

### 5.1.3 Linear Feedback Shift Register (LFSR) IP for RNG

The Linear Feedback Shift Register is a custom hardware block that is used for our games' pseudo-random number generation. This hardware block is specifically used in the Endless Gamemode.

The LFSR is implemented within the programmable logic of the FPGA, and mapped to the system's memory space at a base address (0x43C20000). Our design was an 8-bit LFSR, using 8 Flip Flops and 3 XOR gates, and we could access the value register at another address. Even though we only have an 8-bit LFSR, the AXI-Lite bus operations operate at a width of 32 bits; so the software interface needed to include a bit mask to isolate the 8 bits we are looking for from the rest of the data.

Within the Endlessa game mode, the LFSR is what decides when and how many spikes are generated. The values that can be generated are between 0-255, and we have a preset threshold (set to 0x90 at the moment) to determine how often spikes are generated (this can change to alter the difficulty of the game). If the hardware generated a value that was greater than this preset threshold, a spike would be set to generate; if not, the background would continue to generate. To make sure the game stays possible and playable, the software has two limits: a limiter to make sure no more than 3 spikes can spawn directly back to back (4 spikes is *near* impossible, and 5 is impossible to clear), and a limiter to have a mandatory vuffere following any spike sequence. This makes sure the generated levels are possible and fun to play.

The LFSR needs external input; it simply uses the clock to generate its numbers. Offloading our random number generation to a custom piece of hardware frees our cores from having to constantly generate and poll a new number for the endless game mode. This allows our ARM cores to focus on audio playback as well as rendering graphics to the display as smoothly as possible.

#### 5.1.4 Audio Threshold Detector (ATD) IP

The Audio Threadhold Detector was implemented as a custom AXP piece of hardware IP. This block was needed for the audio game mode, the game mode that generated its obstacles based on the sound amplitudes of the music playing.

This peripheral is integrated into the systems with memory-mapped I/O, and is accessible at its base address. This piece of hardware features three key registers: a Live Value register that captures the current 24-bit audio sample that is going to be processed, a Threshold register that compares the audio amplitude to the pre-set threshold, and the Flag register, which goes high when the live audio crosses the threshold (all of these can be accessed at their memory address). When the Flag register gets set to high, and we read in on the ARM code, we know to tell the video driver software to generate and print a spike.

Code 1 acts as the primary point of contact for the Audio Threadhold IP. Core 1 retrieves a 16-bit stereo sample from the WAV file in DDR memory and then performs a bit-shift conversion to make it into a 24-bit format. Core 1 then pushes this data to the Live Value register on the Audio Threadhold IP. This data then makes its way through the IP's registers until it reaches the flag, then the flag will either be set to high or stay low. To make sure the game stays possible and playable, the software has two limits: a limiter to make sure no more than 3 spikes can spawn directly back to back (4 spikes is *near* impossible, and 5 is impossible to clear), and a limiter to have a mandatory vuffere following any spike sequence. This makes sure the generated levels are possible and fun to play.

When changing to the play state, the Audio Threadhold IP and the software generate the level in seconds. This level of data is stored in on-chip memory at a particular address. To prevent data coherency issues between our dual cores, core 1 uses a cache flush after writing the new level, and core 0 performs a cache invalidate range before rendering. This ensures the processes have the most recent hardware, flagged obstacles, and data.

## 5.2 Software Subsystems

### 5.2.1 Dual-Core Orchestration & Mailbox Synchronization

For our game, we needed to make sure the audio playback does not affect the 60 FPS video rendering, which we needed to make our game playable. For this, we used a Dual Code system and used shared memory pointers to allow the cores to communicate.

Core 0 is the Master; this code handles the primary game logic: the state machine, game physics, collision detection, and all VGA rendering and pixel writing. Core 0 is also the core that responds to the interrupts, and Core 0 actually initializes Core 1.

Core 1 is essentially the audio engine. Code 1 has an isolated main and a while loop. This core's only job is to read the WAV files from DDR, perform data format conversions, and feed them to the Audio Codec hardware for playback.

The two cores communicate through a shared memory pointer, defined in the on-chip memory. This memory region is accessible by both cores, making it global. The global memory is low latency and communicates a path that bypasses the DDR controller.

Our key global shared memory pointers are: `SHARED_AUDIO_STATE_PTR` (0xFFFF0000): A 32-bit control flag used by Core 0 to command Core 1 ( play, stop, or restart audio during a Game Over reset). `GENERATED_MAP_PTR` (0xFFFF2000): A shared buffer used in the audio generated mode, where Core 1 writes the procedurally generated obstacles for Core 0 to render.

Because each ARM core has its own L1 level cache, maintaining data coherence is a must. The on-chip memory section marked as not cachable ensures that writes from one core to another are immediately visible. On the software side, we implemented the `sev()` (send event) assembly instruction to wake up Core 1 from its low-power wait mode. The cache is disabled on Code 1 in our project, this is to allow us to not need to manage the cache between the cores and saves headaches when it comes to coherence.

### 5.2.2 Core 0: Game Engine, Physics, and Collision Detection

Core 0 is the master processor, and it is responsible for almost all of the game logic. This core handles the state machine, the game logic, the physics for jumping, collisions, rotation calculations, and all VGA graphics.

The game physics file handles the jump logic, which renders the icon in steps to animate its jump motion. Gravity was implemented to be used with the jump function. When a jump is pressed using the push button interrupts, a negative vertical acceleration is applied. Each frame, the physics gets updated, and this negative gravity acceleration is applied against the jump strength

variable to simulate jumping up and then coming down. There is a precise delay between jump operations, so a ‘double jump’ is not possible (jumping a second time before hitting the ground).

The player's icon is constrained to a vertical level boundary. This prevents the play from ‘falling’ through the floor, and acts as the level to render in the floor of our level.

For the collision logic, Core 0 performs a pixel alignment check on the hitboxes of our sprite and the spike. A hitbox is the physical region of the spike that cannot overlap. It is standard practice in game development to have the hitbox be an invisible, smaller shape than what the player can see. This is done to make the game more enjoyable and to eliminate the need to be ‘pixel-perfect’ (basically adding wiggle room on the player inputs and reaction times). When Core 0 does its calculations and determines what the two hitboxes have overlapped, the game resets right away. This happens by resetting the player icon to its initial upright and grounded location, and by resetting the array of the current level that is being played (as all of our levels are stored in string arrays consisting of ‘^’ and ‘\_’ for spikes and background blocks. A collision also writes to the shared memory pointer that both cores can read; this is how the audio knows to restart since the play has registered the level.

Core 0 also handles the frame limiter. The frame limiter makes sure that we are staying and writing at a consistent 60 FPS. This is needed because without this, the VGA will write at a much higher FPS when there are no or few obstacles on the screen, then the VGA will slow down when there are multiple elements to render. This slowdown is actually very noticeable, so this limiter ensures that we have a smooth gameplay experience at all times.

### 5.2.3 Core 0: Math-Based Dynamic Video Driver (Replacing SD Images)

Instead of using memcopy and stock images for our menus and pause screen, as we did in the early milestone, we decided to use custom pixel-generated menus to improve the feel of the game and make the menus feel more native. Our video driver software file uses coordinate-based drawing functions to draw out our menus as needed. Colours were set using RRGGBB coloration. Similar to this, the player icon and spikes are obstacles that are generated using

geometry and pixel math. By utilizing our ‘angle’ variable from the physics source file we created, the software can calculate rotated vertex positions in real-time, allowing the icon to perform its signature 90-degree flip during jumps. The spike obstacles are rendered as triangles using a simple algorithm that actually takes a 64x64 pixel square, and highlights the triangle in a unique colour like blue or black, then the rest of that square area with the colour of the background.

For text generation and writing, we used an open-source GitHub repo that provided an 8x8 ASCII font library [1]. This library has the bitmaps for every character we could need. To leverage this, we created a couple of custom helper functions to make writing to the VGA as simple as possible. The first function was the actual ‘draw’ function; this function calls the font8x8 header file and matches the user input with the correct bitmap, then writes it to the VGA (function includes coordinate arguments to ‘place’ the text onto a precise location). The second function is where we take in the characters from the UART (or hardcoded text for elements such as titles), and call the first function to actually ‘draw’ onto the monitor.

For the pause menu, when the pause switch gets flipped up on the FPGA, the VGA freezes its current screen, then loops over all the pixels and reduces the brightness by half, by shifting the GRB value by 1. This gives a ‘darkening’ filter over the current screen. After that, a simple pixel writing algorithm is used to draw a white pause symbol over the screen.

#### 5.2.4 Core 1: SD Card Audio Streaming & Cache Management

The SD card implementation was a change that we made to minimize memory limitations, as storing WAV files can be up to 30 MB per file. So instead of using a .data memcopy for all files, or using multiple large header files, we decided to store all of our audio files onto a 64GB SD card, then dynamically load those audio files as needed to main memory. The audio files are found via file name, and loaded depending on what game mode is being played, or if the user is in the menu. Our custom draw is a wrapper around the FatFS library (xilffs in Vitis), which is Xilinx's port of the open-source FAT file system driver for Zynq. It provides two functions: one to initialize the SD card, and one to load a file from it into a specific memory address. The load

function locates the file by name in the FAT directory and loads the .WAV from the SD card onto FPGA memory.

As mentioned before, the cache is disabled on core 1; this is to maintain cache coherency and to ensure the shared memory pointers do not become race conditions.

## 6.0 Implementation

### 6.1 Design Methodology

For this project, we implemented a hybrid approach for our design methodology. This means we combined a top-down design with a bottom-up implementation. The bottom-up implementation was our priority, as the hardware is really the core part of this project. Not to mention the learning curve for hardware design was much steeper for us compared to the learning curve of the software implementation. On a high level, this design method starts by implementing the core hardware needed for the game: the VGA and audio codec IPs. Then we moved to the top level to write the states of the game. This went back and forth until we finished our game.

Let's go into more detail about how this hybrid method worked. As mentioned, we started by implementing the VGA and audio codec first. This was done by following the provided tutorial. Note that these hardware blocks were created separately at first for individual testing. Next, we moved to the highest level and wrote out our games high level states in C++. This was a simple state machine, with about five basic states. Keyboard presses over UART were used to change states, and a print statement over UART to this Vitis terminal was used to confirm what state we were in, and provide any other details that would have been needed. Once this was done, we made one of our best decisions for this project. We decided to integrate our VGA and audio codec now (early milestone stage). As we expected, this task took much longer and was much more difficult than we originally thought. Combining the hardware took multiple complete rebuilds until we got it right. This best demonstrates the initial hardware design learning curve. Once combined, we moved back to the high level; here we created a simple coloured background, used memcpy and stock images as placeholder menus, and created our player icon, who could jump using push button interrupts. Once this was done, the next few weeks (middle

milestone) were used to create the two custom pieces of hardware. Again, once individual testing was done, we combined and integrated these custom hardware IPs into our A/V hardware setup - we now had all of our hardware complete. Following this, the software section took over: we created the functions and logic for all of our game modes, replaced the memcpy menus with pixel-drawn ones, added a win condition and its screen, high score, and many other bells and whistles since we had the time to do our bonus milestone.

In conclusion, our design methodology leveraged a hybrid approach of combining both top-down and bottom-up design, this allowed us to balance learning hardware design, with our much more advanced software skills. The key decision that made our design methodology the most optimal was early integration and combination of the individual IP blocks, and regular hardware testing

## 6.2 Milestones and Timeline

### Ali-Aryo Otufat-Shamsi's Milestones

| Milestone                                                 | Goal/Description                                                                                   | Key Contributions                                                                                                                                                                                                                                                                                                 |
|-----------------------------------------------------------|----------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Milestone 0: VGA Hardware Implementation</b>           | Creation of a custom VGA controller IP core for video output.                                      | Followed a tutorial given to the class to create a custom VGA controller IP core using Vivado. Implemented precise VGA signal timing (H_SYNC, V_SYNC, RGB signals) and a simple script to display five vertical colour bars and shift their order using push button interrupts.                                   |
| <b>Milestone 1: Overall Game Play States</b>              | Complete high-level system for overall game-play states and template functions.                    | Created all source and header files. Setting up the main state machine (menu, pause, play, gameover/level complete) using UART and print statements.                                                                                                                                                              |
| <b>Milestone 2: Interrupt-Driven Menus and Text Input</b> | Implement switch-based navigation between menus and keyboard interrupts for text input to the VGA. | Added states for the main menu, gameplay, and pause screens triggered by GPIO switch-based interrupts. Created custom helper functions to render text onto the VGA using the open-source font8x8 bitmap font, allowing the player to type characters on the keyboard and from UART to be displayed on the screen. |
| <b>Milestone 3: Auto Scroller and Console Collision</b>   | Implement a hardcoded level auto-scroller and detect                                               | Wrote a helper function to pixel-render and auto-shift spikes (obstacles) across                                                                                                                                                                                                                                  |

| Milestone                                                  | Goal/Description                                                                                                  | Key Contributions                                                                                                                                                                                                                                                                                                          |
|------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                                            | collision, printing the result to the console.                                                                    | the screen. Developed a simple pixel software collision detection function using "hitboxes" to compare the user sprite and obstacle. Detection simply involves printing a statement via UART.                                                                                                                              |
| <b>Milestone 4: LFSR Hardware RNG</b>                      | Create a hardware-based pseudo-random number generator (LFSR) for an alternate game mode.                         | Designed and packaged a custom 8-bit Linear Feedback Shift Register (LFSR) IP block using Vivado. This IP block communicates with the Zynq system via an AXI4-Lite slave bus.                                                                                                                                              |
| <b>Milestone 5: Obstacle Generation and Final Gameplay</b> | Generate obstacles based on LFSR pseudo-random numbers to complete the endless game mode.                         | Implemented the endless gamemode by constantly reading the 0-255 value from the LFSR and comparing it against a programmable threshold to see when to generate spikes. Added software logic to limit consecutive spikes to 3 and set a minimum of eight empty tiles between spikes, to make sure gameplay is fun and fair. |
| <b>Bonus Milestone 1: Main Menu and Pause GUI</b>          | Implementation of a custom Main Menu GUI upon booting and a Pause Menu GUI triggered by an FPGA switch interrupt. | Created custom pixel-drawn graphics for the main menu. For the pause menu, implemented logic that freezes the VGA, applies a 'darkening' filter (reducing GRB brightness by half by shifting the RRGGBB values by 1),                                                                                                      |

| Milestone                         | Goal/Description                                               | Key Contributions                                                                                                                                                                                                          |
|-----------------------------------|----------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                                   |                                                                | and draws a white pause symbol over the screen.                                                                                                                                                                            |
| <b>Bonus Milestone 2: SD Card</b> | Implement SD card software to store and read audio .WAV files. | Developed SD card software to read audio .WAV files from off-board memory onto FPGA memory. Created a wrapper around the FatFS library. Functions to initialize the SD card and load files into a specific memory address. |

### Artin Jafari Rad's Milestones

| Milestone                                                          | Key Accomplishment                                                  | Details of Implementation                                                                                                                                                                                                                                 |
|--------------------------------------------------------------------|---------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Milestone 0: Audio Codec Integration</b>                        | Established the fundamental audio interface.                        | Configured hardware for reliable I2S communication, implementing the I2C control sequence at a 48 kHz sampling rate. Allowing for stable streaming of live audio.                                                                                         |
| <b>Milestone 1: Hardware Merging (VGA &amp; Audio Integration)</b> | Integrated two major subsystems and architected a dual-core design. | Integrated Audio IP blocks and the ADAU1761 codec interface into the Programmable Logic with the VGA design. The system was architected for a dual-core distribution, dedicating Core 1 as the audio engine and Core 0 for VGA rendering and game logic.  |
| <b>Milestone 2: Push-Button Interrupts &amp; UI Navigation</b>     | Implemented responsive user input and core UI structure.            | Developed hardware interrupt logic for GPIO push-buttons for the "jump" mechanic, using a dirty rectangle rendering method to maintain a high frame rate. Mapping FPGA switches to handle menu swaps and implementing software debouncing.                |
| <b>Milestone 3: Hardware Audio Threshold Detection (ATD)</b>       | Created the hardware feature for audio-reactive gameplay.           | Developed a custom hardware IP to analyze the raw 24-bit live audio stream and identify amplitude peaks. This hardware generates a flag that makes a bridge using AXI-Lite to the software to dynamically render a level map based on musical transients. |

| Milestone                                                          | Key Accomplishment                                             | Details of Implementation                                                                                                                                                                                                                                                                            |
|--------------------------------------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Milestone 4: Rhythmic Synchronization &amp; Collision Logic</b> | Finalized playability and essential game-over conditions.      | Refined obstacle generation with "guardrails" to prevent impossible gameplay scenarios. Finalized the pixel-level collision module, which monitored the icon's hitbox's coordinate space against obstacles and triggered a visual signal and reset upon collision.                                   |
| <b>Milestone 5: Full System Integration &amp; Results Display</b>  | Ensured system-wide consistency and robustness across content. | Implemented logic for obstacle generation. Writing a mandatory buffer of no spikes, just after a spike or group of spikes gets generated. Added a mechanism to adjust the sensitivity of the hardware Audio Threshold Detector to ensure consistency when loading different tracks from the SD card. |
| <b>Bonus Milestone: Sprite Selector</b>                            | Added customization to the Sprite.                             | Developed a sprite selector module in the main menu, which dynamically loads the selected sprite's pixel data into the frame buffer by modifying the VGA drawing routines.                                                                                                                           |
| <b>Bonus Milestone: Rotating Jump</b>                              | Implemented dynamic visual effects for the player icon.        | Implemented a dynamic rotation system by tracking a <i>sprite_angle</i> variable and applying a 2D rotation matrix to the sprite's pixel coordinates using a math-based C++ drawing                                                                                                                  |

| Milestone | Key Accomplishment | Details of Implementation                                                                                        |
|-----------|--------------------|------------------------------------------------------------------------------------------------------------------|
|           |                    | <p>approach. Implemented a pixel reading update to synchronize the collision hit-box with the rotating icon.</p> |

## 6.3 Design Changes

Our original idea was to upload the audio file for our levels and menus as either a .data file or as a file header. In practice, these ideas did not seem optimal; they either did not work(.data) or took up too much of the FPGA memory. Due to this, we decided to implement SD software to read the .WAV file from off-board memory, transfer it to FPGA memory, and play it back via the audio codec. The source file for this in core 0 is a wrapper around the FatFS library (xilffs in Vitis), which is Xilinx's port of the open-source FAT file system driver for Zynq. It provides two functions: one to initialize the SD card, and one to load a file from it into a specific memory address. The load function locates the file by name in the FAT directory and loads the .WAV from the SD card onto FPGA memory.

Another improvement we made was that before, we were using a simple memcopy and stock images to be the stand-in for the main menu and the pause menu. Since we finished our core five milestones before the project was due, we had time to implement our bonus milestones. We decided to create a custom main menu and pause menu. The main menu showcases our game (Geometry Dash FPGA edition), as well as has text that instructs what each state is and which switch to be switched to get to those states (states such as the three game modes, name select, and sprite select). The main menu is pixel-drawn: simple square and rectangle shapes for the background, text using our milestone two functions, and calling our different sprites to showcase all of the work we have done. For the pause menu, when the pause switch gets flipped up on the FPGA, the VGA freezes its current screen, then loops over all the pixels and reduces the brightness by half, by shifting the GRB value by 1. This gives a 'darkening' filter over the current screen. After that, a simple pixel writing algorithm is used to draw a white pause symbol over the screen.

## 7.0 Difficulty & Project Complexity

The biggest challenge we faced in this project was in managing the shared resources of the Zedboard. The AXI interconnect became a point of heavy traffic when the VGA and Audio attempted to access the DDR simultaneously. We noticed this in the form of visible screen tearing and interruption of live audio, and the cores ran together. To help with this, we disabled the cache in core 1 and implemented a shared pointer between the two cores, allowing for better communication. Core 0 was dedicated to the heavy lifting of the physics and VGA rendering, whilst core 1 took the entire role of the audio. Synchronization was an issue at first; however, utilizing the shared flag, we were able to correctly interrupt core 1 when needed.

Furthermore, the tuning of the audio threshold mode posed a great challenge. Ensuring the beats were synchronized to the actual output audio as the map was generated posed. We bypassed this by processing the audio map and passing it along to core 1 as the level is loading, always looking a bit ahead for the map generation, so as not to misalign the audio and video. As well, because audio signals vary significantly in range, a static threshold in the PL logic would often result in either over-triggering (creating too many obstacles) or missing beats entirely. We had to develop a software-assisted threshold routine that allowed us to pass real-time parameters to the ATD IP. Finding the sweet spot where the hardware could accurately distinguish the high peaks of the song from background noise required hours of iterative testing with different music genres and songs.

## 8.0 References

[1] D. Hepper, “font8x8\_basic.h,” GitHub Repository. Available:

[https://github.com/dhepper/font8x8/blob/master/font8x8\\_basic.h](https://github.com/dhepper/font8x8/blob/master/font8x8_basic.h)

[2] AMD Xilinx, “UG585: Zynq-7000 SoC Technical Reference Manual,” AMD Documentation.

Available: <https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/>

[3] W. Knitter, “Dual ARM Hello World on Zynq using Vitis,” Hackster.io. Available:

<https://www.hackster.io/whitney-knitter/dual-arm-hello-world-on-zynq-using-vitis-9fc8b7>

[4] Digilent, “ZedBoard: Creating Custom IP Cores,” Digilent Reference. Available:

<https://digilent.com/reference/learn/programmable-logic/tutorials/zedboard-creating-custom-ip-cores/start>

[5] H. Rosenfeldt, “Zynq: Read a WAV-file from SD-card and play it on the audio codec.”

Available:

<https://www.harald-rosenfeldt.de/2017/12/31/zynq-read-a-wav-file-from-sd-card-and-play-it-on-the-audio-codec/>

[6] *The Zynq Book Tutorials for Zybo and ZedBoard,*

[7] ENSC 452 Course Material, “VGA Tutorial,” Simon Fraser University.

[8] Mohammad Shahidzadeh Asadi, SFU

*provided procedural support throughout all stages of the project in the lab, including troubleshooting and testing activities*